

Requirements Engineering

Alsalamah Zaid

Mk 106760

Smart Task Manager

Autor: Zaid Alsalamah

Datum: 13/05/2026

Projektbeschreibung / Vision

Bei dem Projekt handelt es sich um die Entwicklung einer konsolenbasierten Anwendung zur Aufgabenverwaltung (*Smart Task Manager*). Die Anwendung ermöglicht es Nutzern, Aufgaben zu erstellen, zu verwalten, zu priorisieren und als erledigt zu markieren.

Ziel ist es, eine einfache und übersichtliche Lösung zur Organisation von persönlichen oder studienbezogenen Aufgaben bereitzustellen.

Die Umsetzung erfolgt in Java unter Verwendung objektorientierter Programmierung. Die Anwendung bietet grundlegende Funktionen wie das Erstellen, Anzeigen, Sortieren und Löschen von Aufgaben.

Das Projekt dient zudem dazu, Methoden des Requirements Engineering, der UML-Modellierung sowie des Testings im Rahmen des Softwaretechnik-Moduls praktisch anzuwenden.

Werkzeug A (Notion):

Notion interface showing a table titled "Smart Task Manager Requirements Management". The table lists requirements (REQ-01 to REQ-08) with columns for ID, kurz Beschreibung, Beschreibung, Priorität, Status, Kategorie, Autor, Abnahmekriterium, Abhängigkeit, and Historie. The status column uses color-coded tags: "In Bearbeitung" (red), "Offen" (yellow), and "Nicht-funktional" (orange).

Req ID	kurz Beschreibung	Beschreibung	Priorität	Status	Kategorie	Autor	Abnahmekriterium	Abhängigkeit	Historie
REQ-01	Benutzer kann Aufgabe erstellen	Der Benutzer kann eine neue Au	Hoch	In Bearbeitung	Funktional	Benutzer	Aufgabe wird erfolgreich gespe	-	Version 1.0 erstellt
REQ-02	Aufgabe löschen	Der Benutzer kann eine bestehe	Mittel	Offen	Funktional	Benutzer	Aufgabe wird aus der Liste entf	REQ-01	Version 1.0 erstellt
REQ-03	Aufgaben anzeigen	Das System zeigt alle gespeich	Hoch	Offen	Funktional	Entwickler	Aufgabenliste wird korrekt ange	REQ-01	Version 1.0 erstellt
REQ-04	Aufgaben priorisieren	Aufgaben können nach Priorität	Mittel	Offen	Funktional	Projektidee	Aufgaben werden korrekt sortie	REQ-01	Version 1.0 erstellt
REQ-05	Aufgabe abschließen	Benutzer kann eine Aufgabe als	Hoch	Offen	Funktional	Benutzer	Status der Aufgabe wird auf "Eri	REQ-01	Version 1.0 erstellt
REQ-06	Aufgaben nach Datum sortieren	Aufgaben werden nach Fälligkeit	Mittel	Offen	Funktional	Entwickler	Aufgaben erscheinen in korrekt	REQ-01	Version 1.0 erstellt
REQ-07	Systemleistung	Das System soll schnell reagiere	Hoch	Offen	Nicht-funktional	Benutzer	Antwortzeit unter 2 Sekunden	-	Version 1.0 erstellt
REQ-08	Benutzerfreundlichkeit	Die Benutzeroberfläche soll einf	Mittel	Offen	Nicht-funktional	Benutzer	Benutzer kann System ohne Anl	-	Version 1.0 erstellt

Werkzeug B (Jira):

Jira

For you

Recent

Starred

Apps

Plans

Spaces

Recent

My Operations Team

More spaces

Recommended

Create a roadmap

Filters

Dashboards

Teams

Customize sidebar

Search

Create

See plans

Spaces

My Operations Team

Summary Timeline Board Calendar List Forms Development Archived work items Docs Shortcuts

Search board

Filter

Group

ANFORDERUNGEN. 6

REQ-03

KAN-6

REQ-04

KAN-7

REQ-05

KAN-8

REQ-06

KAN-9

REQ-07

KAN-10

REQ-08

IN BEARBEITUNG 1

REQ-02

KAN-5

ABGESCHLOSSEN 1

REQ-01

KAN-4

You've created "KAN-45" work item

View Copy link

Spaces

My

Sum

Q Sea

ANFOR

REQ-C

✓ KA

REQ-C

✓ KA

REQ-C

✓ KA

REQ-C

✓ KA

REQ-C

✓ KA

REQ-C

✓ KA

Add epic / ☒ KAN-6

REQ-03

+

...

Description

Das System zeigt alle gespeicherten Aufgaben übersichtlich im Konsoleninterface an

Subtasks

...

📅

+

0% Done

Work	Pri...	As...	Status
KAN-22 Aufgabenliste auslesen	= M	👤 U	ANFORDERUNGEN. ▾
KAN-23 Formatierte Ausgabe erzeugen	= M	👤 U	ANFORDERUNGEN. ▾
KAN-24 Priorität und Status anzeigen	= M	👤 U	ANFORDERUNGEN. ▾
KAN-25 Leere Liste behandeln (keine Aufgaben vorhanden)	= M	👤 U	ANFORDERUNGEN. ▾

Linked work items

Add linked work item

ZA

Add a comment...

working on this...?

Status update...

Thanks...

to comment

🔒

👁 1

🔗

⋮

↶

✕

Anforderungen. ▾ ⚡ ✖ Improve Task

Details

⚙

Assignee

👤 Unassigned

Assign to me

Priority

🔴 High

Parent

None

Due date

None

Labels

None

Team

None

Start date

None

Reporter

ZA

Zaid Alsalamah

> Development

> Automation ⚡

Rule executions

Created 28 minutes ago

Updated 17 minutes ago

⚙ Configure

Vergleich der verwendeten Werkzeuge

In diesem Projekt habe ich zwei verschiedene Werkzeuge zur Verwaltung der Requirements verwendet: ein einfaches Tool und ein professionelles Tool.

Als einfaches Werkzeug habe ich Notion genutzt. Notion ist leicht zu bedienen und eignet sich gut, um Requirements in einer Tabelle zu organisieren. Man kann schnell neue Anforderungen erstellen und Attribute wie Priorität oder Status hinzufügen. Für kleinere Projekte ist das sehr praktisch. Allerdings fehlen einige fortgeschrittene Funktionen wie komplexe Workflows oder eine direkte Verbindung zur Entwicklung.

Als professionelles Werkzeug habe ich Jira verwendet. Jira ist etwas komplizierter, bietet aber mehr Funktionen. Man kann Requirements als Issues anlegen, diese miteinander verknüpfen und verschiedene Status verwenden. Außerdem kann man den Fortschritt besser verfolgen. Für größere Projekte ist Jira deshalb besser geeignet.

Insgesamt habe ich gemerkt, dass Notion einfacher und schneller zu benutzen ist, während Jira mehr Möglichkeiten bietet, aber auch mehr Einarbeitung braucht.

Dokumentation der Anforderungen

Für das Projekt „Smart Task Manager“ wurden insgesamt 8 Anforderungen (REQ-001 bis REQ-008) dokumentiert. Dabei wurde darauf geachtet, grundlegende Anforderungsmerkmale wie Eindeutigkeit, Identifizierbarkeit und Prüfbarkeit zu berücksichtigen.

Um Missverständnisse zu vermeiden und eine bessere Nachvollziehbarkeit zu gewährleisten, wurde jede Anforderung mit mehreren Attributen beschrieben.

Verwendete Attribute sind:

- **ID & Datum:** Zur eindeutigen Identifizierung und zeitlichen Einordnung der Anforderungen.
- **Autor:** Zur Nachverfolgbarkeit, wer die Anforderung erstellt hat.
- **Kurzbeschreibung:** Kurze und verständliche Zusammenfassung der Anforderung.
- **Beschreibung:** Genauere Erklärung der Funktion mit klarer Formulierung.
- **Kategorie:** Unterscheidung zwischen funktionalen und nicht-funktionalen Anforderungen.
- **Priorität:** Bewertung der Wichtigkeit (z. B. Hoch, Mittel, Niedrig).
- **Status:** Aktueller Stand der Anforderung (z. B. Offen, In Bearbeitung, Erledigt).
- **Abnahmekriterium:** Eine prüfbare Bedingung, die festlegt, wann eine Anforderung als erfüllt gilt.
- **Abhängigkeit:** Zeigt, ob eine Anforderung von anderen abhängt.
- **Historie:** Dokumentation von Änderungen oder Versionen.

Durch diese strukturierte Beschreibung wird eine gute Grundlage für die weitere Entwicklung, das Design sowie das Testen der Anwendung geschaffen.

Mission Statement

Mission

Der Smart Task Manager hat das Ziel, eine einfache und effiziente Lösung zur Verwaltung von Aufgaben bereitzustellen.

Die Anwendung soll Nutzern helfen, ihre täglichen oder studienbezogenen Aufgaben zu organisieren, zu priorisieren und den Fortschritt zu verfolgen.

Der Fokus liegt auf einer klaren Struktur, einfacher Bedienung und schneller Interaktion über eine Konsolenanwendung.

Roadmap

Roadmap

Iteration 1

- Grundstruktur des Projekts erstellen
- Task-Klasse implementieren
- Aufgaben erstellen und anzeigen

Iteration 2

- Aufgaben löschen
- Aufgaben als erledigt markieren
- Sortierung nach Priorität

Iteration 3

- Sortierung nach Datum
- Verbesserung der Benutzerführung
- Vorbereitung von Tests (JUnit)

Tech-Stack

Tech Stack

Programmiersprache

- Java

Architektur

- Objektorientierte Programmierung (OOP)

Tools

- Notion (Requirements Management)
- Jira (Professionelles Tool)
- Git / GitHub (Versionierung)

Testing

- JUnit (Unit Tests)

Entwicklungsumgebung

- IntelliJ IDEA oder Eclipse